

Cours 1 : Rappel

- Algorithme
- Variables
- Enchaînement d'instruction
 - Séquence
 - Condition
 - Répétition
 - Fonction
 - Variable d'une fonction

Algorithme (1/2)

- Un **algorithme** peut prendre de nombreuses formes :
 - Recette de cuisine
 - Notice de montage d'un meuble
 - Itinéraire de ville A vers ville B
 - ...

Algorithme (1/2)

- Un **algorithme** peut prendre de nombreuses formes :
 - Recette de cuisine
 - Notice de montage d'un meuble
 - Itinéraire de ville A vers ville B
 - ...
- A fait la description d'un enchaînement d'actions destinées à être exécutées par B pour résoudre un problème de manière systématique.
 - La description doit être claire, complète et sans ambiguïté.
- Cette description est une forme d'**algorithme**.

Algorithme (1/2)

- Un **algorithme** peut prendre de nombreuses formes :
 - Recette de cuisine
 - Notice de montage d'un meuble
 - Itinéraire de ville A vers ville B
 - ...

- A fait la description d'un enchaînement d'actions destinées à être exécutées par B pour résoudre un problème de manière systématique.
 - La description doit être claire, complète et sans ambiguïté.

- Cette description est une forme d'**algorithme**.

- B est un ordinateur
 - les consignes doivent être donner dans un langage interprétable par l'ordinateur.
 - ces consignes écrites dans un langage de programmation, s'appelle un **programme**.

Algorithme (2/2)

- Des algorithmes sont les expressions **formalisées** des solutions aux problèmes que l'on doit résoudre.
 - Il est difficile et peu souhaitable d'écrire directement un programme pour un ordinateur.
- **Que veut dire formaliser ?**
- 2 exemples de la même question :
 - « Quel nombre faut-il ajouter à 9 pour obtenir 17 ? »
 - l'équation $9 + x = 17$ est sa version formalisée

Exemple : Maximum des 2 nombres

Algorithme : Maximum

/* l'utilisateur donne deux nombres. Il faut afficher le plus grand des deux. */

/* Les déclarations */

Variables

/* identificateur	: type	rôle */
-------------------	--------	---------

nb1, nb2	: entier	/* les deux nombres */
----------	----------	------------------------

maxNb	: entier	/* maximum des 2 nombres */
-------	----------	-----------------------------

/* Les instructions */

Début

1. écrire('Donner deux entiers : ')
2. lire(nb1, nb2)
3. Si nb1 > nb2 alors
4. maxNb ← nb1
- sinon
5. maxNb ← nb2
6. fins
7. écrire('Le plus grand des deux nombres ', nb1, ' et ', nb2, 'est :', maxNb)

Fin

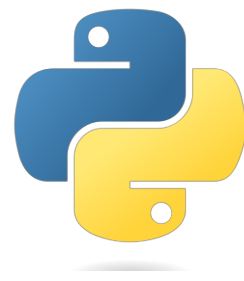
déclarations

instructions

Python



- *Python is*
 - *powerful... and fast;*
 - *plays well with others;*
 - *runs everywhere;*
 - *is friendly & easy to learn;*
 - *is Open.*
 - [<https://www.python.org/about/>]
- Il permet la programmation impérative structurée, fonctionnelle et orientée objet.
- Pendant ce cours on utilise la version 3 du Python.



Qu'est-ce qui caractérise un algorithme/code "bien écrit" ?

Un algorithme, ou code "bien écrit" doit avoir les propriétés suivantes :

- Être facile à lire, par soi-même mais aussi par les autres.
- Avoir une organisation logique et évidente.
- Être explicite, montrer clairement les intentions du développeur.
- Être soigné et robuste au temps qui passe.

Cours 1 : Rappel

- Algorithme
- Variables
- Enchaînement d'instruction
 - Séquence
 - Condition
 - Répétition
 - Fonction
 - Variable d'une fonction

Variables

- Une **variable** est une association entre un nom de la variable et une valeur.
- C'est une zone de mémoire, qui est virtuelle dans un algorithme et physique dans un ordinateur.
- Sa valeur peut changer au cours d'exécution de l'algorithme.
- Un identificateur doit commencer par une lettre ou du tiret souligné, il se compose de lettres, de chiffres et/ou du tiret souligné.
 - Exemple : **sommeFinale** ou **max_entier** ou **max1_entier**
- Chaque variable est aussi caractérisée par son type
 - Une contrainte sur les valeurs qu'elle peut prendre.
 - Si on déclare une variable de type entier, on ne pourra pas y mettre un réel ou un caractère.

Types

Types	en algorithmique	en machine
entier	élément de $\mathbb{Z}$	élément d'un sous ensemble de $\mathbb{Z}$
réel	élément de $\mathbb{R}$	élément d'un sous ensemble de $\mathbb{Q}$
caractère	signe, lettre, chiffre ...	signe, lettre, chiffre avec délimiteur : '
chaîne	suite de caractères	suite de caractère avec délimiteur : "
booléen	Vrai ou Faux	True (1) ou False (0)

Les valeurs de type caractère ou chaîne doivent être entourées par des apostrophes.

Instructions élémentaires

- **Algorithmes** est une suite des instructions (actions) à exécuter.
- **Instruction** prend les variables dans un certain état et les met dans un autre état.
 - A la fin d'exécution d'une instruction certaines variables changent des valeurs
 - Les instructions peuvent être simples (elles s'écrivent sur une ligne) ou structurées (elles renferment d'autres instructions).
- **Instructions élémentaires :**
 - Affectation
 - Lecture
 - Écriture

Affectation

- **Affectation** est une instruction qui modifie la valeur d'une variable à partir d'une expression calculée.
- **Syntaxe :**

< identificateur de variable > ← < expression >

- < expression > est :
 - une variable
 - une constante
 - le résultat d'un calcul en utilisant des constantes et/ou des variables

- **Exemples :**

interet ← (somme * TAUX) / 100

somme ← somme + interet

Lecture

- **Lecture** est une instruction d'entrée qui permet l'interactivité avec l'utilisateur
- **Syntaxe :**

```
lire(< identificateur de variable1>, [<identificateur de variable2>, ...])
```

- **Exemples :**

```
lire(sommeDeCompte)
```

- **Constat :**
 - La valeur de variable est écrasée par la nouvelle valeur
 - Le type de la valeur entrée doit être le même ou compatible avec le type de la variable.

Écriture

- **Écriture** est une instruction de sortie qui aussi permet l'interactivité avec l'utilisateur.
- **Syntaxe :**

```
écrire( <expression_1>, [<expression_2>, ...] > )
```

- **Exemples :**

```
x ← 9
```

```
écrire("le successeur de ", x, " est ", x+1)
```

- **Constat :**
 - L'instruction envoie à l'écran le résultat

Expressions

- Une **expression simple** est :

- une valeur (constante)

10

28.1

't'

- l'identificateur d'une variable (qui donne l'accès à sa valeur) : nb, g ou lettre

nb

g

lettre

158

9.81

h

- la combinaison de valeurs et/ou identificateurs de variables (opérandes) et d'opérateurs :

nb + 10

- Une **expression complexe** : les opérandes peuvent être eux-mêmes des expressions

(nb + 10) * g

Type d'une expression

- Type d'une expression est le type de son résultat
- Exemple :

nb
158

g
9.81

lettre
h

$nb + 10$ est de type ?

$(nb + 10) * g$ est de type ?

'h' est de type ?

Type d'une expression

- Type d'une expression est le type de son résultat
- Exemple :

nb
158

g
9.81

lettre
h

$nb + 10$ est de type **entier**

$(nb + 10) * g$ est de type **réel**

'h' est de type **caractère**

Expressions numériques

- Soit l'expression

a op b

- où **a, b** sont les opérandes et **op** l'opérateur.

	a - entier		a - réel	
b - entier	+ , - , *	→ entier	+ , - , * , /	→ réel
	/	→ réel		
b - réel	+ , - , * , /	→ réel	+ , - , * , /	→ réel

Expressions booléennes

- Une variable booléenne est une variable à deux états : vrai et faux.
- Elle est utilisée à représenter les valeurs de vérité.

- **Exemple :**

 acces : booléen

 acces ← vrai

- Une expression booléenne peut être construite à partir
 - des variables en utilisant les opérateurs de comparaison (< , <= , > , >= , = , <>)
 - d'autres expressions booléennes et des opérateurs booléennes (non , et, ou)

Tables de vérités

- Tables de vérités des opérateurs booléennes :

A	non A
vrai	faux
faux	vrai

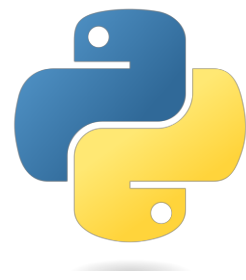
A	B	A et B	A ou B
vrai	vrai	vrai	vrai
vrai	faux	faux	vrai
faux	vrai	faux	vrai
faux	faux	faux	faux

- **Priorité** : non est prioritaire sur et qui est prioritaire sur ou

Expressions booléennes

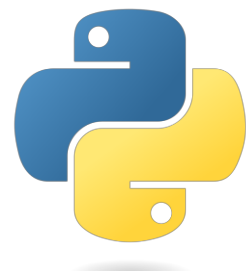
Questions

1. $76 > 90 \rightarrow$ **faux**
2. $x = 5.45$
 $x < 10 \rightarrow$ **vrai**
3. $\text{non}(76 > 90) \rightarrow$ **vrai**
4. $(20 > 3)$ et $(20 < 15)$
vrai et **faux** $\rightarrow$ **faux**
5. $(20 > 3)$ ou $(20 < 15)$
vrai ou **faux** $\rightarrow$ **vrai**
6. $(46 > 10)$ et $(55 < 100)$
? et ? $\rightarrow$?
7. $(46 > 100)$ ou $\text{non}(2 < 10)$
? ou ? $\rightarrow$?



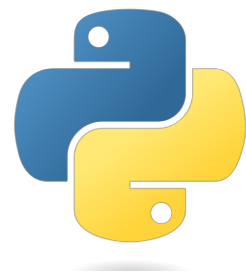
Variables en Python (1/2)

- Les données sont stockées dans des variables.
- Il n'est pas possible définir de constante (une valeur qui n'est pas modifiable au cours d'exécution du programme).
- Une variable est une association entre le nom de la variable et une valeur qui peut varier au cours d'exécution du programme.



Types des variables

- Les types de base en Python sont :
 - **int** : entier
 - **float** : réel
 - **str** : chaîne de caractères
 - **bool** : booléen (prends les valeurs **True (1)** et **False (0)**)
 - **NoneType** : un type spécial qui permette qu'une seule valeur **None** (qui signifie : « pas de valeur » ou « valeur manquante »)

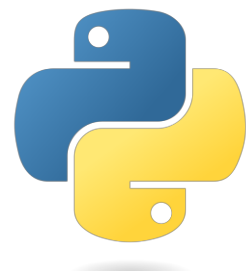


Variables en Python (2/2)

- **Les variables sont typées dynamiquement en Python:**
 - Une variable a un type précis qui lui est attribué au moment de l'exécution d'un programme.
 - Mais ce type peut être modifié au cours de l'exécution du programme.
 - Le type d'une variable n'est pas déclaré par l'utilisateur : il est défini par la valeur effective que l'on décide de stocker dans la variable.

Remarque : C, C++ et Java sont des langages statiquement typés.

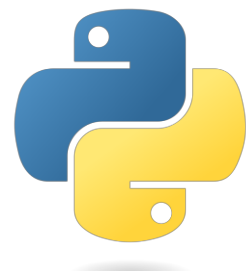
Contraintes pour les noms de variable



- Les noms de variable :
 - doivent débuter par une lettre ou par le symbole _
 - ne doivent contenir que des lettres, des chiffres et des symboles _
 - ne doivent pas correspondre à un mot réservé du Python

```
and del for is raise assert elif from lambda return break else  
global  
not try nonlocal True False class except if or while continue import  
pass yield None def finally in as with
```

- ne doivent pas correspondre à des noms de la librairie standard de Python
- Les noms de variable sont sensibles à la casse



Opération d'affectation

- On utilise l'opérateur = pour affecter une valeur à une variable

- **Exemple :**

x = 5

y = 7

somme = x + y

print(somme)

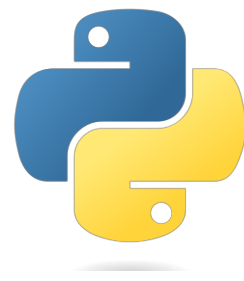
z = 5.7 * x *(x-1)

print(z)

print(type(z))

print("La valeur courante de z est {}".format(z))

```
itkachenko — python3 — 71x24
Last login: Mon Sep 12 10:40:29 on ttys000
[(base) itkachenko@MacBook-Pro-de-Iuliiia ~ % python3
Python 3.9.7 (default, Sep 16 2021, 08:50:36)
[Clang 10.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> x = 5
[>>> y = 7
[>>> somme = x + y
[>>> print(somme)
12
[>>> z = 5.7 * x *(x-1)
[>>> print(z)
114.0
[>>> print("La valeur courante de z est {}".format(z))
La valeur courante de z est 114.0
[>>> print(type(z))
<class 'float'>
[>>> ]
```



Opération de lecture et d'écriture

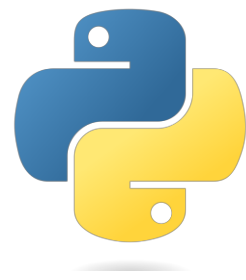
- **Ecriture :**

```
print(z)  
print(type(z))  
print("La valeur courante de z est {}".format(z))
```

- **Lecture :**

```
n1 = int(input())  
n2 = int(input('Entrez un nombre :'))  
somme = n1+n2
```

```
c1 = input()  
c2 = input('Entrez une chaine de caractère')  
c1+c2
```



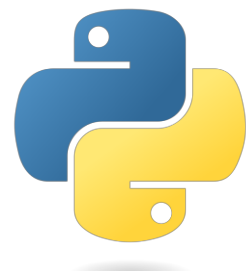
Opération de lecture

▪ Booléenne

```
[>>> b1 = bool(input())
0
[>>> print(b1)
True
[>>> b2 = bool(input())
[>>> print(b2)
False
[>>>
[>>>
[>>> b3 = bool(input())
false
[>>> print(b3)
True
[>>> █
```

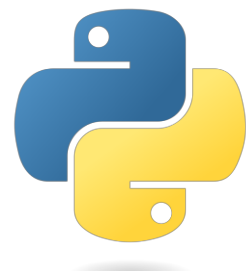
▪ Chaîne de caractères

```
[>>> n1 = input()
5
[>>> n2 = input("Entrez un nombre : ")
Entrez un nombre : 17
[>>> somme = n1 + n2
[>>> somme
'517'
```



Opérateurs (1/3)

Opérateur	Opération
+	Addition
-	Soustraction
*	Multiplication
/	Division
**	Elévation à la puissance
%	Modulo
//	Division entière



Opérateurs (2/3)

Opérateur	Opération
+	Addition
-	Soustraction
*	Multiplication
/	Division
**	Elévation à la puissance
%	Modulo
//	Division entière

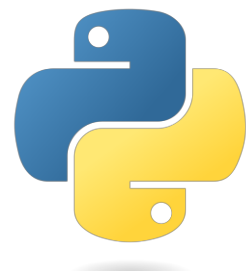
Les opérateurs % et // sont ceux de la division euclidienne :

$$a = b * q + r \text{ avec } r < b$$

$$q = a // b \quad r = a \% b$$

q - quotient entier dans la division entière de a par b

r - reste dans la division entière de a par b



Opérateurs (3/3)

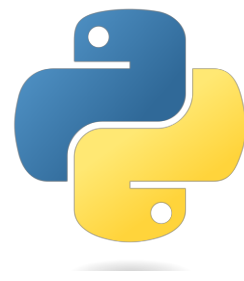
Opérateur	Opération
+	Addition
-	Soustraction
*	Multiplication
/	Division
**	Elévation à la puissance
%	Modulo
//	Division entière

Il existe des opérateurs associés qui réalisent successivement l'opération demandée puis l'affectation de la nouvelle valeur à la variable :

`x = x + 2`

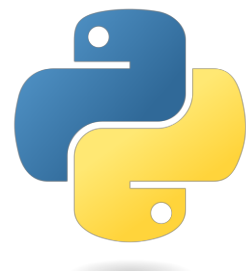
équivalant à

`x += 2`



Priorité d'opérateurs

- Quand l'évaluation d'une expression implique plusieurs opérateurs, les règles de priorité sont appliquées :
 1. Parenthèses
 2. Élévation à la puissance
 3. Multiplication ou division
 4. Addition ou soustraction
 5. De gauche à droite
- Quand une opération est effectuée entre deux variables de types différentes, le type le plus générique est retenu.
 - Exemple : multiplication entre un entier et un nombre à virgule donne les résultats de type **float**



Exercice

- Prédire des résultats des opérations suivantes :

`(1+2)**3`

`"Da" * 4`

`"Da" + 3`

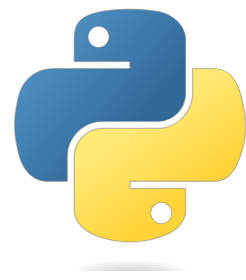
`("Pa"+"La") * 2`

`("Da"*4) / 2`

`5 / 2`

`5 // 2`

`5 % 2`



Exercice : solution

- Prédire des résultats des opérations suivantes :

`(1+2)**3`

`"Da" * 4`

`"Da" + 3`

`("Pa"+"La") * 2`

`("Da"*4) / 2`

`5 / 2`

`5 // 2`

`5 % 2`

```
[>>> (1+2)**3
27
[>>> "Da"*4
'DaDaDaDa'
[>>> "Da"+3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
[>>> ("Pa"+"La")*2
'PaLaPaLa'
[>>> ("Da"*4)/2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for /: 'str' and 'int'
[>>> 5/2
2.5
[>>> 5//2
2
[>>> 5%2
1
[>>> █
```

Cours 1 : Rappel

- Algorithmes
- Variables
- Enchaînement d'instruction
 - Séquence
 - Condition
 - Répétition
 - Fonction
 - Variable d'une fonction

Séquence d'instruction

- La séquence d'instruction est la forme la plus simple d'enchaînement d'instructions.
- Les instructions sont écrites l'une après l'autre, séparées par un saut de ligne.

```
x ← lire("x=")
p2 ← x*x
p3 ← p2*x
p4 ← p3*x
écrire(p2, p3, p4)
```

```
x = float(input("x="))
p2 = x*x
p3 = p2*x
p4 = p3*x
print(p2, p3, p4)
```



Instruction conditionnelle

- **Forme complète d'instruction conditionnelle** permet d'exécuter la première séquence d'instructions si la condition est vraie sinon la deuxième séquence d'instructions est exécutée.

- **Syntaxe :**

```
Si <condition> alors
    <séquence d'instructions 1 >
sinon
    <séquence d'instructions 2 >
finsi
```

- <condition> est évaluée :
 - vrai → la séquence d'instructions 1 est exécutée
 - faux → la séquence d'instructions 2 est exécutée

Exemple : instruction conditionnelle

Début

age ← 12

si age < 18 alors

 écrire('L'âge est inférieur à 18 ans');

sinon

 écrire('L'âge est supérieur à 18 ans');

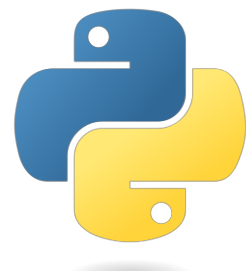
finsi

Fin

Instruction si ... sinonSi ... finSi

- Cette structure conditionnelle est préférable quand il y a beaucoup de si imbriqués
- **Syntaxe :**

```
si <expression booléenne1> alors
    < séquence n°1 >
sinonSi <expression booléenne2> alors
    < séquence n°2 >
[sinonSi <expression booléenne3> alors
    < séquence n°3 >
...
sinonSi <expression booléenneN> alors
    < séquence n°N >]
sinon
< séquence n°N+1 >
finSi
```



Structures conditionnelles (1/2)

- Permet indiquer à un programme de n'exécuter une séquence d'instructions que si une certaine condition est remplie
- **Exemple :**

```
x = 5
```

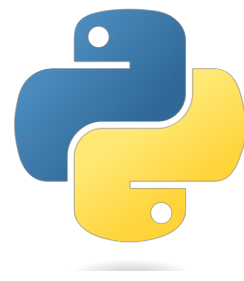
```
if x > 0:
```

```
    print("x est positif")
```

```
else:
```

```
    print("x est négatif")
```

L'indentation est de rigueur
pour chacun des deux blocs
d'instructions



Structures conditionnelles (2/2)

- Il est possible de définir plusieurs comportements en fonction de plusieurs tests successifs à l'aide de la structure sinon si
- **Exemple :**

```
x = -1
```

```
if x > 0:
```

```
    print("x est positif")
```

```
    x = 4
```

```
elif x > -2:
```

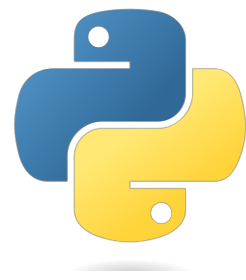
```
    print("x est compris entre -2 et 0")
```

```
elif x > -4:
```

```
    print("x est compris entre -4 et -2")
```

```
else:
```

```
    print("x est inférieur à -4")
```



Opérateurs de comparaison

Opérateur	Comparaison effectuée	Exemple
<	Plus petit que	<code>x < 0</code>
>	Plus grand que	<code>x > 0</code>
<=	Plus petit ou égal à	<code>x <= 0</code>
>=	Plus grand ou égal à	<code>x >= 0</code>
==	Egal à	<code>x == 0</code>
!=	Différent de	<code>x != 0</code>
is	Test d'égalité pour le cas de la valeur None	<code>x is None</code>
is not	Test d'inégalité pour le cas de la valeur None	<code>x is not None</code>
in	Test de présence d'une valeur dans une liste	<code>x in [1, 5, 7]</code>

Cours 1 : Rappel

- Algorithmes
- Variables
- Enchaînement d'instruction
 - Séquence
 - Condition
 - Répétition
 - Fonction
 - Variable d'une fonction

Répéter des calculs

- Nous voulons calculer la somme des 5 premiers entiers naturels :

- **Solution :**

$$\text{somme} = 1 + 2 + 3 + 4 + 5$$

- Et si on désire calculer, non pas la somme des 5 mais des 100 000 premiers entiers naturels ?

- **Solution :**

- On peut formellement faire la même chose : $\text{somme} = 1 + 2 + 3 + 4 + 5 + \dots + 99991$

- Et si on veut maintenant calculer des **N** premiers entiers naturels ? **N** étant variable.

- **Solution:**

- Il est nécessaire de définir **des calculs répétitifs (des boucles)**, dont le nombre d'étapes n'est pas fixé à l'avance

Instruction **TANT QUE**

- **Instruction Tant que** est une boucle qui permet de répéter un bloc d'instructions **tant** qu'une certaine **condition booléenne donnée est satisfaite**.
- **Syntaxe :**

```
tantque <condition> faire  
    <séquence d'instructions>  
fintantque
```

- **Remarques :**
 - Il faut que la condition soit **évaluable**
 - Si la condition est fausse, **la séquence ne sera jamais exécutée**
 - La condition doit être **modifié** par la séquence d'instruction, sinon la boucle sera éternelle

Exercice

- Combien de fois la boucle ci-dessous sera exécuté ?

Début

$a \leftarrow 1$

$b \leftarrow a$

tantque $a < 5$ faire

$b \leftarrow b * 5$

écrire(b)

fintantque

Fin

Exercice : solution

- Combien de fois le boucle ci-dessous sera exécuté ?

Début

$a \leftarrow 1$

$b \leftarrow a$

tantque $a < 5$ faire

$b \leftarrow b * 5$

écrire(b)

fintantque

Fin

a	condition	b
1		1
1	vrai	5
1	vrai	25
1	vrai	125
...		

Boucle éternelle ! $a < 5$ toujours vrai

Instruction Répéter

- **Instruction Répéter** est une boucle qui permet de répéter un bloc d'instructions jusqu'à une certaine condition booléenne donnée est satisfaite.
- **Syntaxe :**

```
répéter  
    <séquence d'instructions>  
jusqu'à <condition>
```

- **Remarques :**
 - La séquence est **exécutée au moins une fois**
 - La séquence doit modifier la condition de sortie, sinon la boucle sera éternelle

Comparaison Tantque et répéter

somme $\leftarrow$ 0

sommeFinale $\leftarrow$ 20

tantque somme < sommeFinale faire

 somme $\leftarrow$ somme + 10

fintanque

somme $\leftarrow$ 0

sommeFinale $\leftarrow$ 20

répéter

 somme $\leftarrow$ somme + 10

jusqu'à somme $\geq$ sommeFinale

Instruction Pour

- **Instruction Pour** est une boucle qui est contrôlée par le comptage du nombre de répétitions de la séquence.
- **Syntaxe :**

```
pour <variable> de <expr_début> à <expr_fin> faire  
    <séquence d'instructions>  
finpour
```

- **Remarques :**
 - La séquence ne doit modifier :
 - ni la variable de boucle
 - ni l'expression de fin
 - La séquence peut ne jamais être exécutée si, avant d'aborder l'instruction pour, <expr_début> est déjà strictement supérieure à <expr_fin>

Comparaison Tantque et Pour

somme $\leftarrow$ 0

pour i de 0 à 5 faire

 somme = somme + i

finpour

somme $\leftarrow$ 0

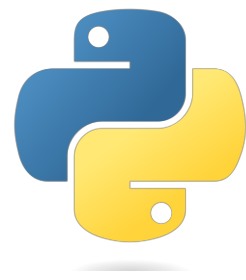
i $\leftarrow$ 0

tantque i $\leq$ 5

 somme = somme + i

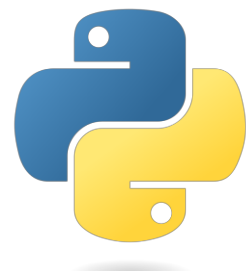
 i $\leftarrow$ i + 1

fintantque



Boucles en Python

- Deux types de boucles :
 - Les boucles qui s'exécutent tant qu'une condition est vraie
 - Les boucles qui répètent la même série d'instructions pour différentes valeurs d'une variable



Boucle Tantque

▪ Exemples :

```
x = 0
```

```
while x <= 10:
```

```
    print(x)
```

```
    x = 3*x + 2
```

Itérations :

0	x = 2
2	x = 8
8	x = 26

```
x = 2
```

```
y = 0
```

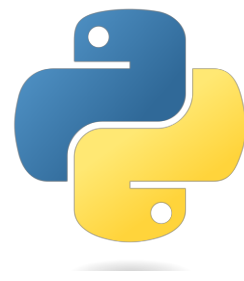
```
while x > 0:
```

```
    y = y - 1
```

```
    print(y)
```

```
print("Si on arrive ici, la boucle est finie")
```

La boucle est éternelle !



Boucle Pour

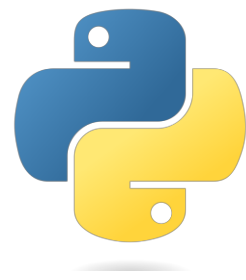
- Ce type de boucle repose sur l'utilisation de listes
 - Une liste est un ensemble ordonné d'éléments.
- On peut exécuter une série d'instructions pour toutes les valeurs d'une liste.
- **Exemples :**

On peut définir les valeurs d'une liste :

```
for x in [1,5,7]:  
    print(x)  
print("Fin de la boucle")
```

Il est possible d'itérer sur les n-1 premiers entiers :

```
n = 10  
for i in range(1,n):  
    print(i)
```



Boucle Répéter

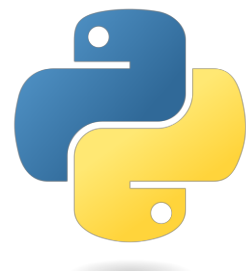
- Il n'existe pas d'instruction `do ... while` en Python
- On peut simuler la boucle Répéter en utilisant la boucle `while`

- **Exemple :**

```
secret_word = "python"  
counter = 0
```

```
while True:
```

```
    word = input("Enter the secret word: ").lower()  
    counter = counter + 1  
    if word == secret_word:  
        break  
    if word != secret_word and counter > 7:  
        break
```



Quelle boucle choisir ?

- On utilisera la boucle **for**
 - Si on souhaite parcourir les éléments d'une liste ou d'un dictionnaire
 - Si on connaît à l'avance le nombre d'itérations qu'on souhaite effectuer
- On utilisera la boucle **while**
 - Si on souhaite répéter une action tant qu'une condition est vérifiée.

Cours 1 : Rappel

- Algorithmes
- Variables
- Enchaînement d'instruction
 - Séquence
 - Condition
 - Répétition
 - Fonction
 - Variable d'une fonction

Sous-programme : motivation

- Nous avons souvent besoin de décomposer un problème global en sous-problèmes :
 - Chaque sous-problème sera géré par une fonction qui pourra elle-même faire appel à d'autres fonctions.
- **Si l'ensemble d'instructions est susceptible d'être utilisée plusieurs fois dans un ou plusieurs programmes, il est recommandé de l'isoler dans un sous-programme (une fonction).**
- Les algorithmes peuvent faire appel à des fonctions :
 - prédéfinies (fonctions mathématiques, fonctions sur les chaînes de caractères, etc.).
 - définies par l'utilisateur en les donnant un identificateur et de programmer les calculs qui conduisent à ce qu'elles doivent renvoyer.

Exemple (1/2)

Algorithme : Maximum_si

Variables

nb1, nb2, nb3 : entier maxNb12, maxNb : entier

Début

lire(nb1, nb2, nb3)

si nb1 > nb2 alors

maxNb12 $\leftarrow$ nb1

sinon

maxNb12 $\leftarrow$ nb2

finsi

si maxNb12 > nb3 alors

maxNb $\leftarrow$ maxNb12

sinon

maxNb $\leftarrow$ nb3

finsi

écrire(maxNb)

Fin

Algorithme : Maximum_fonction

Variables

nb1, nb2, nb3 : entier maxNb12, maxNb : entier

Début

lire(nb1, nb2, nb3)

maxNb12 $\leftarrow$ maxDeux(nb1, nb2)

maxNb $\leftarrow$ maxDeux(maxNb12, nb3)

écrire(maxNb)

Fin

Où maxDeux(x,y) est une fonction définie par l'utilisateur

Exemple (2/2)

Algorithme : Maximum_fonction

Variables

nb1, nb2, nb3 : entier
maxNb12, maxNb : entier

Début

lire(nb1, nb2, nb3)
maxNb12 $\leftarrow$ maxDeux(nb1, nb2)
maxNb $\leftarrow$ maxDeux(maxNb12, nb3)
écrire(maxNb)

Fin

Où maxDeux(x,y) est une fonction définie par l'utilisateur

fonction maxDeux(x, y : entier) : entier

/*retourne le max des deux entier x et y */

Variables locales

max_xy : entier

Début

si x > y alors
 max_xy $\leftarrow$ x
sinon
 max_xy $\leftarrow$ y
finsi
retourner(max_xy)

Fin

Fonction

- Chaque fonction doit avoir un identifiant (son nom) et une liste de paramètres d'entrée.
- **Syntaxe :**

```
fonction <ident-fonction> (<ident-paramètre> : <type>
    ...
    <ident-paramètre> : <type>) : <type>
/* spécification et conditions d'appels */
Variables locales
<ident-variable> : <type>
....
<ident-variable> : <type>

Début
    <instructions>
    retourner <expression>

Fin
```

Passage de paramètres

- Appel d'une fonction

Début

```
...  
maxNb12 ← maxDeux(200,150)  
...
```

Fin

Paramètres effectifs (ou réels)

- Définition d'une fonction

fonction maxDeux(x, y : entier) : entier

Variables locales

max_xy : entier

Début

si x > y alors

max_xy ← x

sinon

max_xy ← y

finsi

retourner(max_xy)

Fin

Paramètres formels

Passage de paramètres

- Appel d'une fonction

Début

...

maxNb12 ← maxDeux(200,150)

...

Fin

Paramètres effectifs (ou réels)

- Définition d'une fonction

fonction maxDeux(x, y : entier) : entier

Variables locales

max_xy : entier

Début

si x > y alors

max_xy ← x

sinon

max_xy ← y

finsi

retourner(max_xy)

Fin

une correspondance biunivoque entre les paramètres formels et les paramètres effectifs - l'ordre d'écriture

Paramètres formels

Passage de paramètres

- Appel d'une fonction

Début

```
...  
maxNb12 ← maxDeux(200,150)  
...
```

Fin

- Définition d'une fonction

fonction maxDeux(x, y : entier) : entier

Variables locales

max_xy : entier

Début

si x > y alors

max_xy ← x

sinon

max_xy ← y

finsi

retourner(max_xy)

Fin

Procédures : motivations

- Procédures sont aussi des blocs d'instructions référencés par un nom et qui ont une spécification précise. On peut dire que c'est un autre type de sous-programme.
- Une **fonction** renvoie une valeur qu'elle a calculé à partir des paramètres reçus.
- Une **procédure** exécute des instructions en utilisant des valeurs reçues en paramètres, mais elle peut aussi modifier ces valeurs. Les paramètres ont un statut (un paramètre d'entrée et/ou un paramètre de sortie).

Procédure

- Chaque procédure doit avoir un identifiant (son nom) et une liste de paramètres (d'entrée at/ou de sortie).
- **Syntaxe :**

```
procédure <ident-fonction> (<statut><ident-paramètre> : <type>
```

```
...
```

```
    <statut><ident-paramètre> : <type>)
```

```
/* spécification et conditions d'appels */
```

```
Variables locales
```

```
<ident-variable> : <type>
```

```
....
```

```
<ident-variable> : <type>
```

```
Début
```

```
    <instructions>
```

```
Fin
```

Procédure avec paramètres d'entrée / sortie

Algorithme tri_lettres

Début

lire(c1, c2, c3)

classer(c1, c2)

classer(c2, c3)

classer(c1, c2)

écrire(c1, c2, c3)

Fin

procédure classer(Entrée/Sortie a, b : caractère)

Début

si $a > b$ alors

échanger(a, b)

finsi

Fin

procédure échanger(Entrée/Sortie x, y : caractère)

Variables locales

aux : caractère

Début

$\text{aux} \leftarrow x$; $x \leftarrow y$; $y \leftarrow \text{aux}$

Fin

Procédure avec paramètres d'entrée et paramètres de sortie

Algorithme duréeConvertie

Variables

durée: entier /*en minute*/

h, min : entier

Début

lire(durée)

convertir(durée, h, min) écrire(h, min) Fin

Fin

procédure convertir(Entree dur : entier

Sortie h : entier

Sortie min : entier)

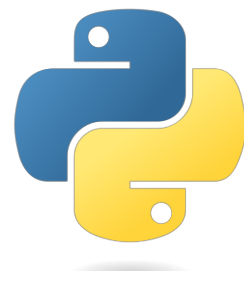
Début

$h \leftarrow \text{dur} \text{ div } 60$

$\text{min} \leftarrow \text{dur} \text{ mod } 60$

Intérêt d'utiliser une fonction

- Ne pas dupliquer le code
- Donner une meilleure lisibilité (si on choisit un nom de fonction informative)
- Partager le développement entre différentes équipes
- Construire des bibliothèques de fonction pour réutiliser



Fonctions en Python

- Pour définir une fonction en Python, on utilise le mot-clé **def**.

- **Exemple :**

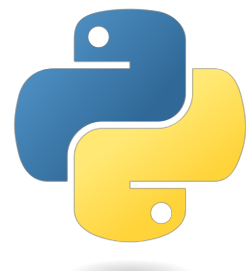
```
def f(x):
```

```
    y = 5 * x + 2
```

```
    z = x + y
```

```
    return z // 2
```

- ou **f** est un nom de la fonction, **x** est un argument (paramètre) de la fonction,
- Cette fonction retourne une valeur indiquée dans la ligne qui débute par le mot-clé **return**

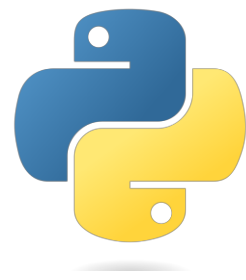


Retour de plusieurs valeurs

- Il est possible d'écrire des fonctions retournant plusieurs valeurs.
- Ces valeurs doivent être séparées par des virgules dans l'instruction **return**
- **Exemple :**

```
def f(x):  
    y = 5 * x + 2  
    z = x + y  
    return z // 2, y
```

- **Remarque:**
 - Une fonction retourne la valeur **None**, si l'instruction **return** est absente



Utilisations des arguments

- Il est possible d'utiliser le nom des arguments de la fonction lors de l'appel, pour ne pas risquer de se tromper dans l'ordre des arguments.

- **Exemple :**

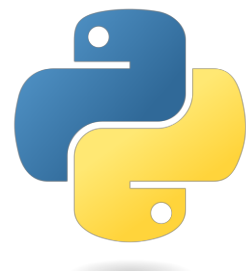
```
def affiche_infos_personne(poids, taille):  
    print("Poids: ", poids)  
    print("Taille: ", taille)
```

- Les 3 appels de la fonction équivalents :

```
affiche_infos_personne(80, 180)
```

```
affiche_infos_personne(taille=180, poids=80)
```

```
affiche_infos_personne(poids=80, taille=180)
```



Arguments optionnels

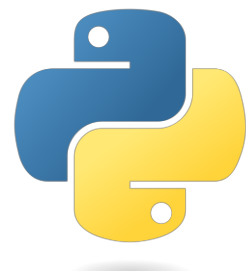
- Certains arguments d'une fonction peuvent avoir une valeur par défaut choisit par le développeur
- Si l'utilisateur ne spécifie pas explicitement de valeur pour ces arguments lors de l'appel à la fonction, c'est la valeur par défaut qui sera utilisée dans la fonction.
- **Exemple :**

```
def f(x, y=0):  
    return x + 5 * y
```

- **Remarque :** les arguments facultatifs doivent impérativement se trouver, dans la liste des arguments, après le dernier argument obligatoire.

```
def f(x, y=0, z):  
    return x - 2 * y + z
```

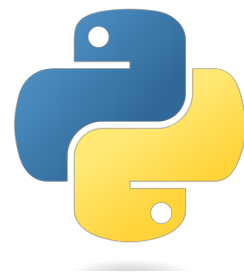
Erreur !!!



Exemple : utilisation des arguments de la fonction

```
def nb_chiffres(n: int) -> int:
    """
    :pré-cond: n > 0
    :post-cond: retourne le nombre de chiffres nécessaires
    pour écrire n en base 10
    """
    c = 1
    while n > 10:
        c = c+1
        n = n//10
    return c
```

```
def total_chiffres_fact(n: int) -> int:
    """ :pré-cond: n > 0
    :post-cond: retourne le nombre total de chiffres nécessaires
    pour écrire les factorielles de tous les entiers entre 1 et n
    """
    f = 1
    c = 0
    for i in range(1, n+1):
        f = f*i
        c = c+nb_chiffres(f)
    return c
```



Exemple : utilisation des arguments de la fonction

Python Tutor: Visualize code in [Python](#), [JavaScript](#), [C](#), [C++](#), and [Java](#)

Python 3.6
[known limitations](#)

```
→ 1 def nb_chiffres(n: int) -> int:
  2     """
  3     :pré-cond: n > 0
  4     :post-cond: retourne le nombre de chiffres néc
  5                 écrire n en base 10
  6     """
  7     c = 1
  8     while n > 10:
  9         c = c+1
 10         n = n//10
 11     return c
12
13 def total_chiffres_fact(n: int) -> int:
14     """
15     :pré-cond: n > 0
16     :post-cond: retourner le nombre total de chiffr
17                 écrire les factorielles de tous le
18     """
19     f = 1
20     c = 0
21     for i in range(1, n+1):
        Edit this code
```

→ line that just executed
→ next line to execute



<< First < Prev Next > Last >>

Step 1 of 58

Frames

Objects